CS-3180: Comparative Languages Lab Problem 7: Safety and Ownership

PURPOSE: The goal of this assignment is to introduce systems programming concepts in Rust. You will build a small word and line counting command-line utility (`wc`).You will experiment with data ownership and memory allocation. You will also practice type inference and pattern matching.

DIRECTIONS: You will implement your project in a binary project named `wc`.Your program will be mimicing the unix `wc` command. Your project will count the number or lines or words present in a file(s). You must structure your CLI argument parsing around an explicit `Mode` enum type.

Mode: You must include the following enum in your code to track user provided flags:

```rust
#[derive(PartialEq, Copy, Clone)]
enum Mode {
    WORDS,
    LINES,
}
```

Here, these `derive` traits amount to:

- **PartialEq**: Let's you use `==`
- **Copy**: Implicity, cheap data copying
- **Clone**: Explicit, expensive data copying (`.clone()`)

CLI Specifications

Arguments:

- Default (no flags provided): Count total **words** across all specified file paths.
- `-l` flag provided: Count total **lines** across all specified file paths. If the `-l` flag is not the first argument, display an error and exit cleanly.
- `[FILES]` required list of files to count the word (or lines) in.

Files and Error:

- Read target files safely using `std::fs::read_to_string`. Propagate any errors cleanly using the `?` operator or `Result` handling instead of panicking with `.unwrap()`.

Counting:

- Word count mode must split input on whitespace and count total words present.
- Line count mode must count total line breaks (using newline characters is sufficient, but be sure to test edge cases).
- Hint: `String` has a function called `split_whitespace()`

Example Output

EXAMPLE: Sample usage

```
> cargo run -- -l file1.txt file2.txt
24
> cargo run -- sample.txt
12
> cargo run -- -l sample.txt
3
> cargo run -- sample.txt -l
Usage: counter [-l] <files...>
```

RUBRIC:

(80 pts) **Base Functionality**

[15] Mode enum definition is properly integrated

[20] CLI arguments are parsed correctly and invalid flag placement exits cleanly

[25] Files are read safely and error propagation handles missing files cleanly

[20] Word and line counting accurately outputs across multiple files

(20 pts) **Style**

[10] Idiomatic ownership transfers and matching

[10] Completed lab report in Markdown format

IMPORTANT NOTES:

- **Submissions that do not compile will receive a zero**
- **Submissions with improperly cited AI will receive a zero and an academic integrity violation**
- **Submissions that are partially or fully copied from another submission will receive a zero and an academic integrity violation**